



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERIA

Escuela Profesional de Ingeniería de Sistemas

Proyecto Motor de Integridad Multibase NoSQL

Curso: Base de Datos II

Docente: Ing. Patrick José Cuadros Quiroga

Integrantes:

Salas Acuña, Adrian Fabricio (2021069829)

Lopez Ojeda, Keyla Noelia (2023078690)

**Tacna – Perú
2026**



Datastack



CloudNexus

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	KLO / ASA	ASA	KLO	01/10/2026	Versión original



Datastack · CloudNexus S.A.C.

Sistema Motor de Integridad Multibase NoSQL

Documento de Especificación de Requerimientos de Software

Versión 1.0

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	KLO / ASA	ASA	KLO	01/10/2026	Versión original

ÍNDICE GENERAL

INTRODUCCIÓN	4
I. Generalidades de la Empresa	4
1. Nombre de la Empresa	4
2. Visión	4
3. Misión	4
4. Organigrama	5
II. Visionamiento de la Empresa	5
1. Descripción del Problema	5
2. Objetivos de Negocios	6
3. Objetivos de Diseño	6
4. Alcance del proyecto	6
5. Viabilidad del Sistema	7
6. Información obtenida del Levantamiento de Información	7
III. Análisis de Procesos	8
a) Diagrama del Proceso Actual – Diagrama de actividades	8
b) Diagrama del Proceso Propuesto – Diagrama de actividades Inicial	8
IV. Especificación de Requerimientos de Software	9
a) Cuadro de Requerimientos funcionales Inicial	9
b) Cuadro de Requerimientos No funcionales	10
c) Cuadro de Requerimientos funcionales Final	11
d) Reglas de Negocio	13
V. Fase de Desarrollo	13
1. Perfiles de Usuario	13
2. Modelo Conceptual	14
a) Diagrama de Paquetes	14
b) Diagrama de Casos de Uso	15
c) Escenarios de Caso de Uso (narrativa)	16
3. Modelo Lógico	22
a) Análisis de Objetos	22
b) Diagrama de Actividades con objetos	23
c) Diagrama de Secuencia	25
d) Diagrama de Clases	29
CONCLUSIONES	30
RECOMENDACIONES	30
BIBLIOGRAFÍA	31
WEBGRAFÍA	31

INTRODUCCIÓN

El presente documento especifica los requerimientos del software *Motor de Integridad Multibase NoSQL*, que la empresa Datastack desarrolla para CloudNexus S.A.C. Su finalidad es describir de forma completa, verificable y sin ambigüedades lo que el sistema debe hacer y las condiciones bajo las cuales debe operar, de modo que sirva como acuerdo entre el cliente y el equipo de desarrollo y como base para el diseño de la arquitectura (FD04), la construcción y las pruebas.

La especificación sigue las buenas prácticas de la norma ISO/IEC/IEEE 29148:2018 para la ingeniería de requisitos (International Organization for Standardization [ISO], 2018) y emplea el Lenguaje Unificado de Modelado (UML) para representar los procesos, los casos de uso y el modelo lógico del sistema (Object Management Group [OMG], 2017). El documento retoma el problema y los objetivos definidos en el documento de visión (FD02) y los resultados del informe de factibilidad (FD01).

I. Generalidades de la Empresa

1. Nombre de la Empresa

CloudNexus S.A.C. es una empresa tacneña dedicada a brindar a pequeñas y medianas empresas una plataforma de comercio electrónico y logística en la nube. Su plataforma integra la gestión del catálogo de productos, de los clientes, de los pedidos y de los pagos, y se apoya en una arquitectura de persistencia políglota con MongoDB, Redis y Apache Cassandra.

2. Visión

Ser la plataforma de comercio electrónico en la nube de referencia para las pymes del sur del Perú, reconocida por la confiabilidad de su información y la calidad de su servicio.

3. Misión

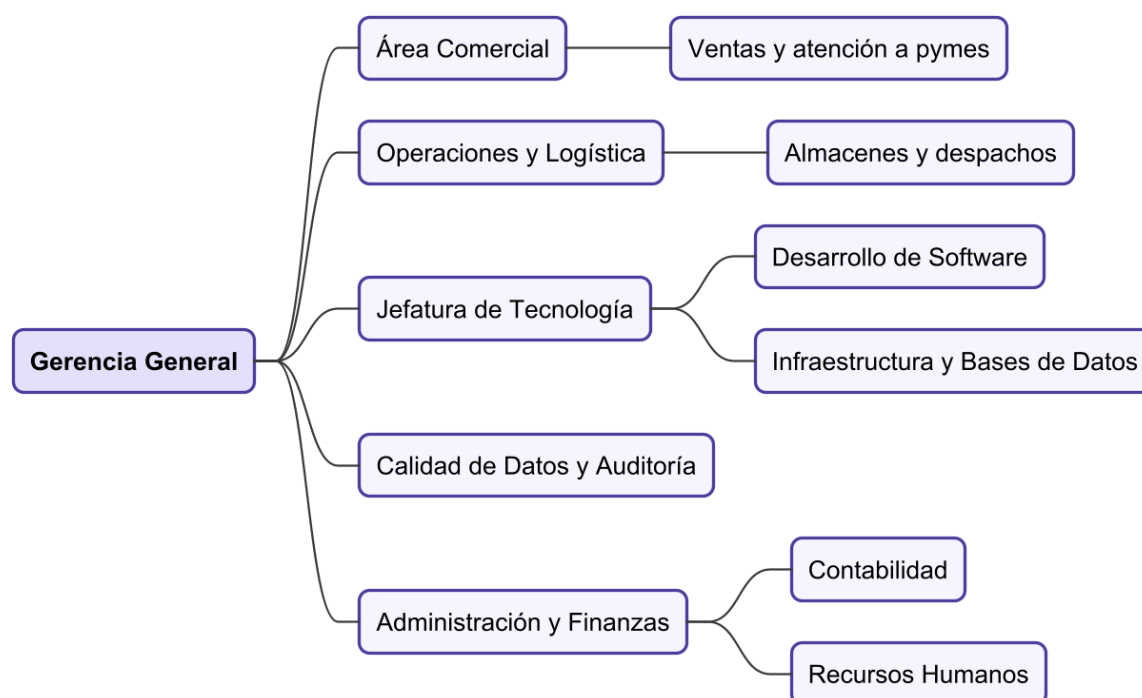
Brindar a las pequeñas y medianas empresas soluciones de comercio electrónico y logística en la nube, seguras, escalables y fáciles de usar, que les permitan vender más y tomar mejores decisiones con datos confiables.

4. Organigrama

La Figura 1 muestra la estructura organizacional de CloudNexus. Las áreas que participan directamente en el proyecto son la Jefatura de Tecnología y el área de Calidad de Datos y Auditoría.

Figura 1

Organigrama de CloudNexus S.A.C.



Nota. Elaboración propia con base en la información proporcionada por CloudNexus.

II. Visionamiento de la Empresa

1. Descripción del Problema

CloudNexus almacena sus datos en tres motores NoSQL que no imponen restricciones de integridad entre ellos: MongoDB trabaja con un esquema flexible por defecto (MongoDB, Inc., s.f.), Apache Cassandra no tiene claves foráneas ni integridad relacional (The Apache Software Foundation, s.f.) y Redis almacena valores sin un esquema declarado (Redis Ltd., s.f.). En consecuencia, aparecen documentos con estructura incorrecta, clientes duplicados, pagos asociados a pedidos inexistentes y datos replicados que divergen entre motores.

Hoy estas anomalías se detectan tarde y se revisan con consultas y scripts manuales que consumen unas 40 horas mensuales del personal de tecnología, cuyos resultados se consolidan en hojas de cálculo sin evidencia auditable. Este problema es común en las arquitecturas de persistencia políglota, en las que la responsabilidad de la integridad recae en las aplicaciones (Sadallage y Fowler, 2012).

2. Objetivos de Negocios

- Reducir en al menos 75 % las horas mensuales dedicadas a revisar manualmente la integridad de los datos.
- Disminuir las pérdidas por errores de datos (reembolsos, facturación errónea y reprocesos).
- Contar con evidencia auditable de la calidad de los datos para auditorías internas y externas.
- Aumentar la confianza de las pymes clientes en la información de pedidos y pagos.

3. Objetivos de Diseño

- Separar la interfaz, el control y la lógica de negocio mediante el patrón Modelo-Vista-Controlador.
- Acceder a los motores mediante adaptadores con una interfaz común, para aplicar las mismas reglas a MongoDB, Redis y Cassandra y facilitar la incorporación de nuevos motores.
- Centralizar usuarios, reglas, conexiones y auditoría en un plano de control (Supabase) protegido con RLS.
- Proteger las credenciales con cifrado Fernet y las contraseñas con bcrypt.
- Desplegar la aplicación en la nube sin costo de infraestructura y accesible desde cualquier navegador.

4. Alcance del proyecto

El sistema permite registrar usuarios, gestionar las conexiones a MongoDB, Redis y Apache Cassandra, definir reglas de esquema, unicidad, integridad referencial y

consistencia, validar registros antes de escribirlos, ejecutar escaneos por lotes, registrar las violaciones en un historial de auditoría, monitorear los motores y consultar los resultados en un dashboard y en reportes PDF. No incluye transacciones distribuidas, la corrección automática de los datos, la intercepción de escrituras de aplicaciones de terceros ni el soporte de otros motores NoSQL.

5. Viabilidad del Sistema

El informe de factibilidad (FD01) concluyó que el proyecto es viable en sus dimensiones técnica, económica, operativa, legal, social y ambiental. El costo total de desarrollo es S/ 11 721.67 y el precio de implementación para CloudNexus es S/ 14 000.00. Desde la perspectiva del cliente, la inversión es rentable: la relación beneficio/costo es 1.38, el VAN es S/ 9 057.58 y la TIR es 46.97 %, superior al costo de oportunidad de capital del 12 %.

6. Información obtenida del Levantamiento de Información

Para identificar las necesidades se aplicaron las técnicas que resume la Tabla 1. Sus hallazgos dieron origen a los requerimientos iniciales de la sección IV.

Tabla 1

Técnicas de Levantamiento de Información y Hallazgos

Técnica	Participantes	Hallazgos principales
Entrevista	Jefatura de Tecnología	Los tres motores no comparten reglas; las credenciales se guardan en archivos locales; se requiere una herramienta en la nube sin instalación.
Entrevista	Área de Calidad de Datos y Auditoría	Se necesitan reglas de unicidad y de referencias entre pedidos, pagos y clientes, y reportes con evidencia para las auditorías.
Observación	Administradores de BD	La revisión manual toma unas 40 horas al mes y los resultados se registran en hojas de cálculo.
Revisión documental	Equipo de Datastack	Documentación oficial de MongoDB, Redis y Cassandra: ninguno ofrece integridad referencial ni validación entre motores.
Prueba de concepto	Equipo de Datastack y CloudNexus	Con datos de prueba se detectaron siete violaciones de los cuatro tipos de regla en los tres motores.

Nota. Elaboración propia.

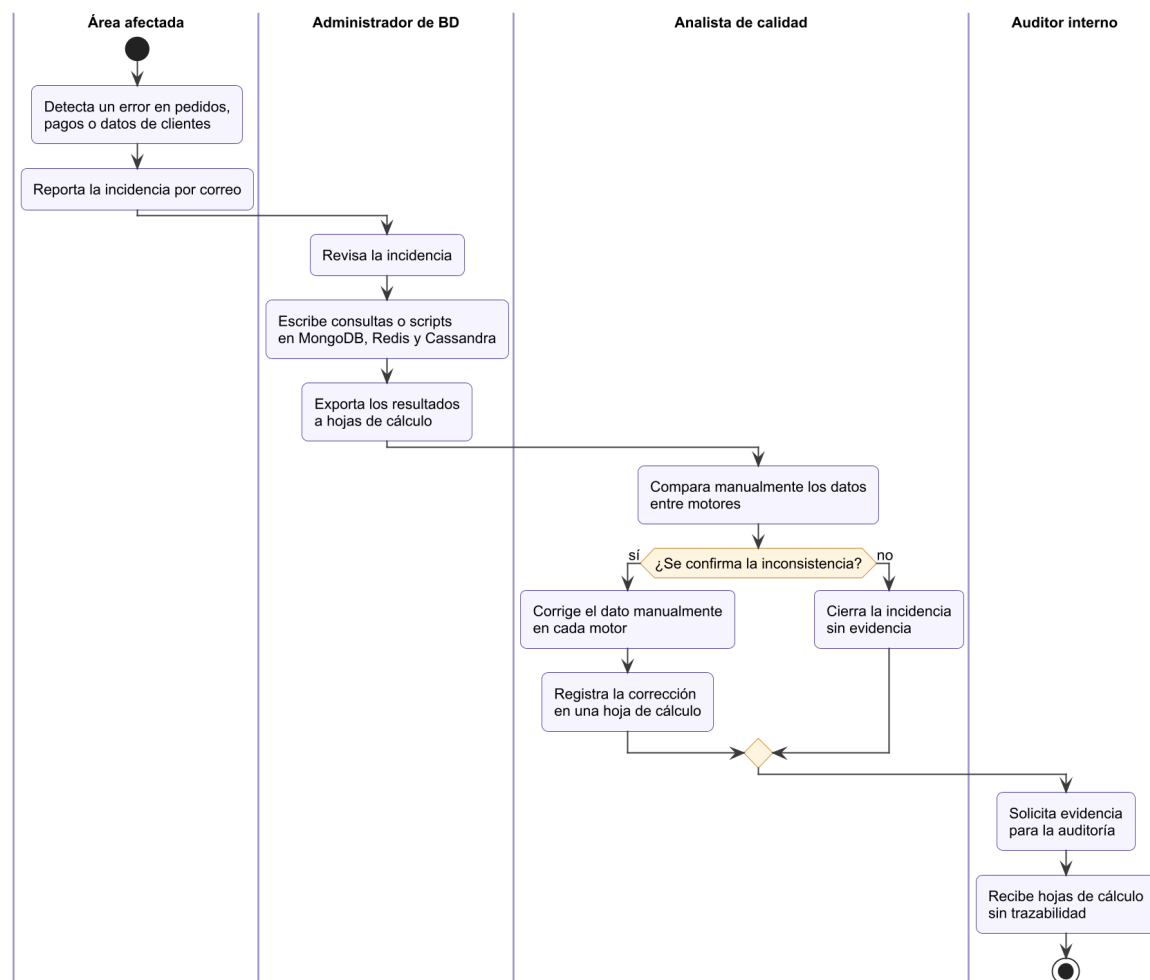
III. Análisis de Procesos

a) Diagrama del Proceso Actual – Diagrama de actividades

La Figura 2 representa el proceso actual de control de la integridad de los datos en CloudNexus. El proceso es reactivo: comienza cuando un área detecta un error, depende de consultas manuales en cada motor y termina sin evidencia trazable para la auditoría.

Figura 2

Proceso Actual de Control de la Integridad de los Datos



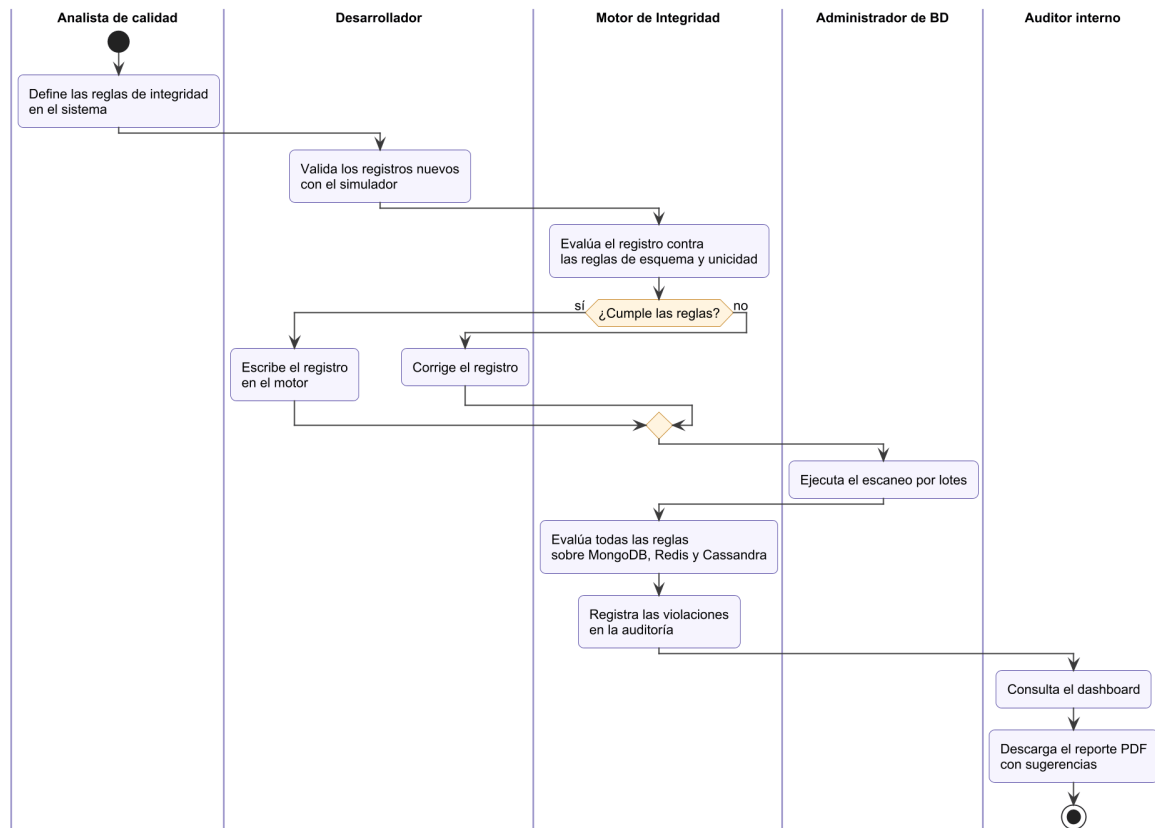
Nota. Elaboración propia con PlantUML (The PlantUML Team, s.f.).

b) Diagrama del Proceso Propuesto – Diagrama de actividades Inicial

La Figura 3 muestra el proceso propuesto con el Motor de Integridad. El control pasa a ser preventivo, con la validación previa a la escritura, y detectivo, con el escaneo por lotes; las violaciones quedan registradas y el auditor las consulta en el dashboard y en reportes PDF.

Figura 3

Proceso Propuesto con el Motor de Integridad Multibase NoSQL



Nota. Elaboración propia con PlantUML (The PlantUML Team, s.f.).

IV. Especificación de Requerimientos de Software

a) Cuadro de Requerimientos funcionales Inicial

La Tabla 2 recoge los requerimientos funcionales tal como fueron expresados por los interesados durante el levantamiento de información.

Tabla 2
Requerimientos Funcionales Iniciales

ID	Requerimiento	Fuente
RFI01	Permitir el acceso al sistema solo a usuarios registrados.	Jefatura de Tecnología
RFI02	Registrar la conexión a las bases de datos MongoDB, Redis y Cassandra.	Administrador de BD
RFI03	Definir reglas que validen la estructura de los datos.	Calidad de Datos
RFI04	Detectar registros duplicados.	Calidad de Datos
RFI05	Verificar que los pedidos y pagos referencien registros existentes en otra base.	Auditoría
RFI06	Comparar los datos que se replican entre motores.	Jefatura de Tecnología
RFI07	Probar un registro antes de guardarlo en la base de datos.	Desarrollo
RFI08	Revisar todos los datos existentes cuando se necesite.	Administrador de BD
RFI09	Ver un resumen de los errores encontrados y métricas generales.	Gerencia General
RFI10	Obtener un reporte descargable de los errores.	Auditoría
RFI11	Conocer el estado de las bases de datos.	Administrador de BD

Nota. Elaboración propia.

b) Cuadro de Requerimientos No funcionales

La Tabla 3 especifica los requerimientos no funcionales, que describen los atributos de calidad que el sistema debe cumplir.

Tabla 3
Requerimientos No Funcionales

ID	Atributo	Descripción	Prioridad
RNF01	Seguridad	Las contraseñas se almacenan con hash bcrypt y nunca en texto plano.	Alta
RNF02	Seguridad	Las credenciales de los motores se almacenan cifradas con Fernet y las tablas de Supabase tienen seguridad a nivel de fila (RLS).	Alta
RNF03	Seguridad	Cada usuario accede únicamente a sus conexiones, reglas y violaciones.	Alta
RNF04	Comunicaciones	Toda comunicación usa HTTPS o TLS.	Alta
RNF05	Rendimiento	La validación de un registro responde en menos de 3 segundos.	Media
RNF06	Fiabilidad	Si una regla no puede evaluarse, el escaneo continúa con las demás y conserva sus violaciones anteriores.	Alta
RNF07	Usabilidad	La interfaz está en español y un usuario nuevo crea una regla y ejecuta un escaneo en menos de 10 minutos.	Media
RNF08	Compatibilidad	Funciona en las versiones actuales de Chrome, Edge y Firefox sin instalación.	Media
RNF09	Disponibilidad	La aplicación está disponible las 24 horas, sujeta a la disponibilidad de los proveedores en la nube.	Media
RNF10	Mantenibilidad	La arquitectura sigue el patrón MVC y el 100 % de las pruebas unitarias se aprueban antes de cada publicación.	Alta
RNF11	Extensibilidad	Se puede incorporar un nuevo motor implementando la interfaz Adaptador sin modificar el motor de reglas.	Media
RNF12	Portabilidad	La aplicación se despliega en Streamlit Community Cloud con Python 3.12.	Media

Nota. Elaboración propia.

c) Cuadro de Requerimientos funcionales Final

Tras el análisis, los requerimientos iniciales se refinaron, se dividieron cuando correspondía y se vincularon con los casos de uso. La Tabla 4 presenta los requerimientos funcionales finales.

Tabla 4
Requerimientos Funcionales Finales

ID	Requerimiento	Descripción	Prioridad	CU
RF01	Registrar cuenta	Crear una cuenta con usuario único (3 a 30 caracteres, sin distinguir mayúsculas) y contraseña de al menos 6 caracteres protegida con bcrypt.	Alta	CU01
RF02	Iniciar sesión	Autenticar al usuario con su usuario y contraseña y abrir su espacio de trabajo.	Alta	CU02
RF03	Cerrar sesión	Finalizar la sesión y volver a la pantalla de acceso.	Media	CU03
RF04	Registrar conexión	Registrar o actualizar la conexión a MongoDB, Redis o Cassandra, probarla y guardarla cifrada con Fernet.	Alta	CU04
RF05	Eliminar conexión	Eliminar la conexión guardada de un motor.	Media	CU04
RF06	Crear regla	Crear una regla de esquema, unicidad, integridad referencial o consistencia, con motor, colección y campo de origen y, si corresponde, de destino o JSON Schema.	Alta	CU05
RF07	Listar reglas	Mostrar las reglas activas del usuario.	Alta	CU05
RF08	Eliminar regla	Eliminar una regla junto con sus violaciones registradas.	Media	CU05
RF09	Validar registro	Validar un registro JSON contra las reglas de esquema y unicidad de su colección antes de escribirlo e informar el resultado.	Alta	CU06
RF10	Ejecutar escaneo por lotes	Recorrer los datos de los motores y evaluar todas las reglas activas del usuario.	Alta	CU07
RF11	Registrar violaciones	Guardar cada violación con su regla, tipo, motores involucrados, dato afectado y fecha, reemplazando las de la ejecución anterior de la misma regla.	Alta	CU08
RF12	Mostrar dashboard	Mostrar los motores utilizados, la cantidad de reglas y de anomalías y su distribución.	Media	CU10
RF13	Diagnosticar motor	Mostrar el estado de conexión, la latencia, la versión y los recursos de un motor.	Media	CU09
RF14	Explorar datos del motor	Mostrar esquemas, una muestra de registros y las estadísticas del servidor.	Baja	CU09
RF15	Generar reporte de anomalías	Listar las violaciones con su explicación y una sugerencia de corrección.	Media	CU11
RF16	Exportar reporte PDF	Descargar el reporte de anomalías en formato PDF.	Media	CU12
RF17	Mostrar usuarios en línea	Mostrar los usuarios con actividad en los últimos cinco minutos.	Baja	CU13

Nota. Elaboración propia.

d) Reglas de Negocio

Las reglas de negocio de la Tabla 5 restringen el comportamiento del sistema y deben respetarse en todos los casos de uso.

Tabla 5

Reglas de Negocio

ID	Regla de negocio
RN01	El nombre de usuario es único sin distinguir mayúsculas: «Ana» y «ana» son la misma cuenta.
RN02	La contraseña debe tener al menos 6 caracteres.
RN03	Cada usuario solo puede ver y modificar sus propias conexiones, reglas y violaciones.
RN04	Un usuario tiene como máximo una conexión por motor; registrarla de nuevo la actualiza.
RN05	Una regla es de uno de cuatro tipos: esquema, unicidad, referencial o consistencia.
RN06	Las reglas referencial y de consistencia exigen motor, colección y campo de destino.
RN07	Una regla de esquema exige un JSON Schema válido y evalúa el registro completo.
RN08	En la unicidad, el primer registro con un valor se considera válido y los siguientes, duplicados.
RN09	Un campo vacío o nulo no se considera valor: no genera duplicados ni referencias huérfanas.
RN10	En la consistencia, los registros enlazados deben existir en ambos motores y coincidir en los atributos que comparten.
RN11	Cada escaneo reemplaza las violaciones anteriores de cada regla evaluada; si la regla falla, se conservan las anteriores.
RN12	Al eliminar una regla se eliminan sus violaciones registradas.
RN13	Un usuario se considera en línea si tuvo actividad en los últimos cinco minutos.

Nota. Elaboración propia.

V. Fase de Desarrollo

1. Perfiles de Usuario

La Tabla 6 describe los perfiles que interactúan con el sistema, incluidos los actores de sistema.

Tabla 6

Perfiles de Usuario y Actores

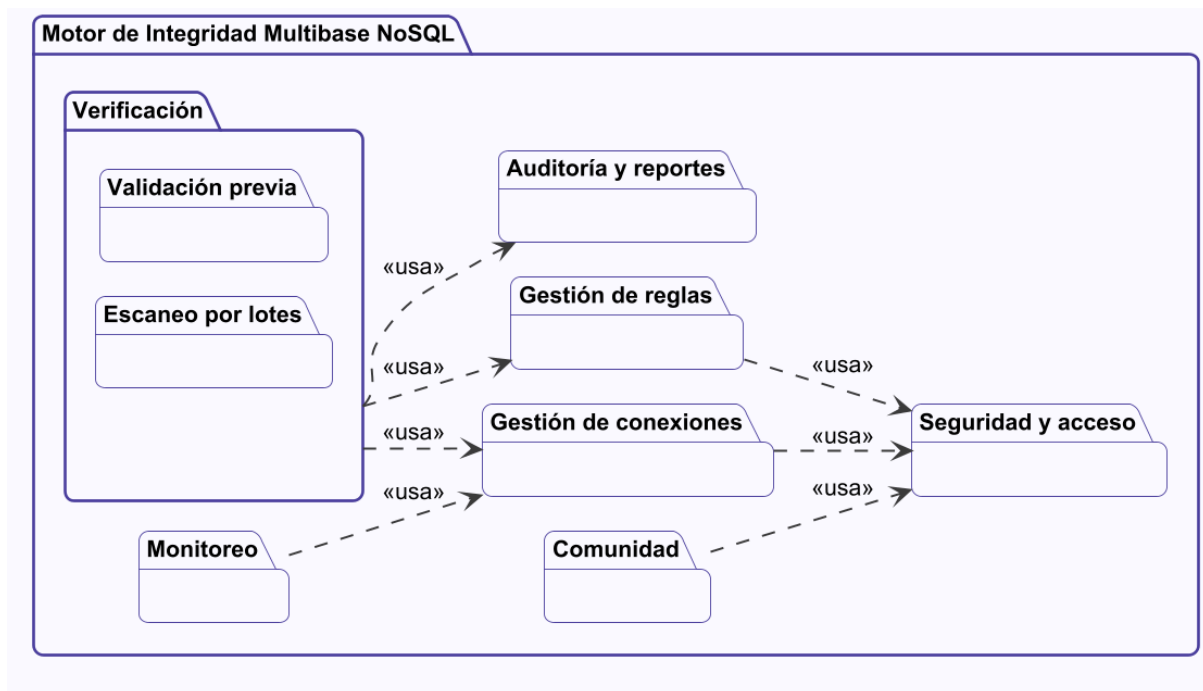
Perfil o actor	Descripción
Visitante	Persona que aún no inició sesión; puede registrarse o iniciar sesión.
Usuario registrado	Actor general autenticado; hereda los casos de uso comunes.
Administrador de BD	Gestiona las conexiones, monitorea los motores y ejecuta escaneos.
Analista de calidad de datos	Define las reglas, ejecuta escaneos y revisa anomalías.
Desarrollador	Valida registros antes de escribirlos.
Auditor interno	Consulta el dashboard y descarga reportes.
Supabase	Actor de sistema: plano de control que almacena usuarios, reglas, conexiones y auditoría.
Motores NoSQL	Actor de sistema: MongoDB Atlas, Upstash Redis y DataStax Astra.

Nota. Elaboración propia.

2. Modelo Conceptual

a) Diagrama de Paquetes

El sistema se organiza en los paquetes funcionales de la Figura 4. La verificación depende de la gestión de reglas, de la gestión de conexiones y de la auditoría, y todos los paquetes se apoyan en el de seguridad y acceso.

Figura 4
Diagrama de Paquetes del Sistema


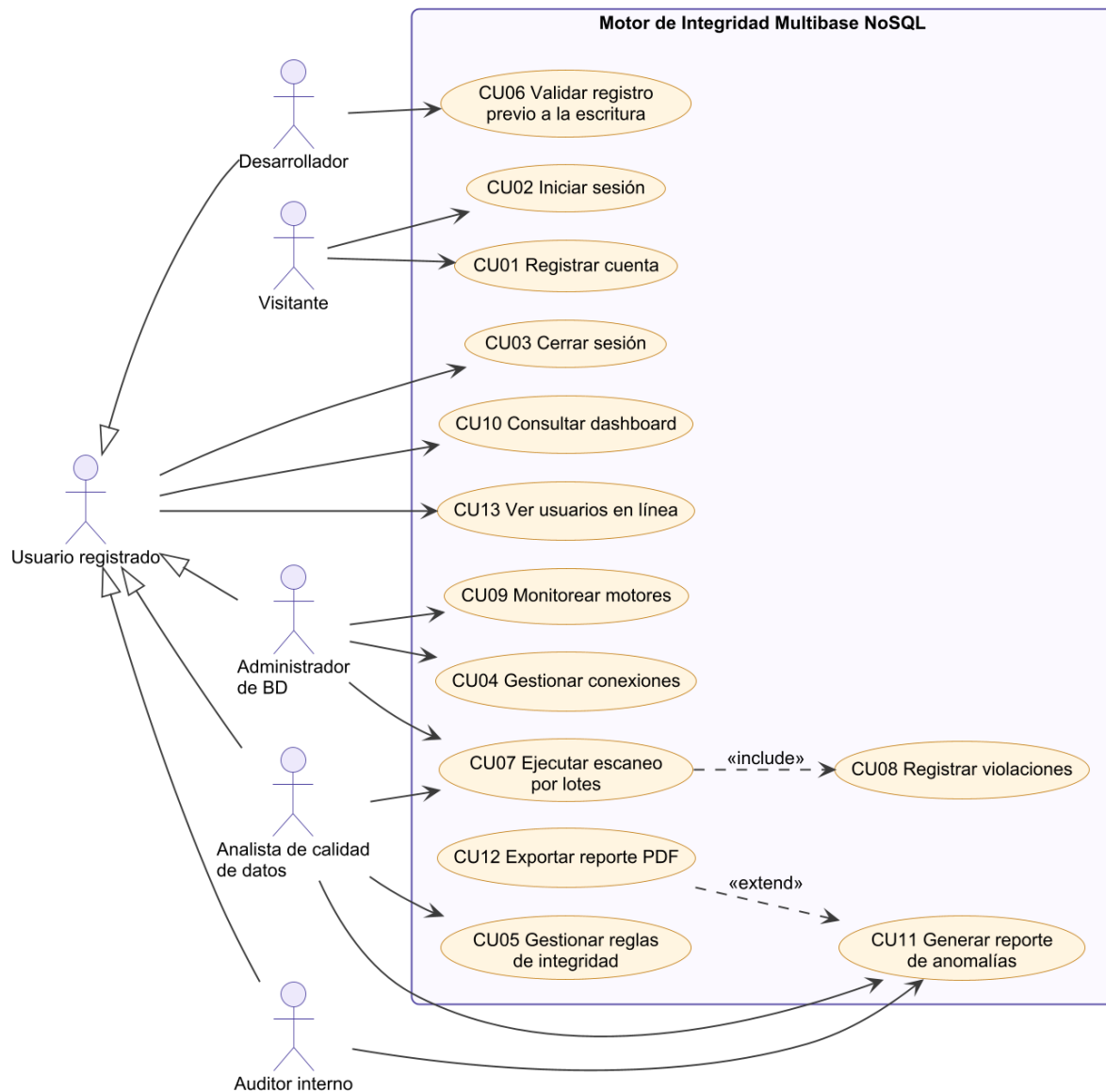
Nota. Elaboración propia con PlantUML (The PlantUML Team, s.f.).

b) Diagrama de Casos de Uso

La Figura 5 presenta los trece casos de uso del sistema. Los actores especializados heredan los casos de uso del usuario registrado; el escaneo por lotes incluye el registro de violaciones y la exportación PDF extiende la generación de reportes. Supabase y los motores NoSQL participan como actores de sistema en los casos que almacenan o consultan datos.

Figura 5

Diagrama de Casos de Uso del Motor de Integridad Multibase NoSQL



Nota. Elaboración propia con PlantUML (The PlantUML Team, s.f.).

c) Escenarios de Caso de Uso (narrativa)

Las Tablas 7 a 19 describen la narrativa de cada caso de uso, siguiendo el formato de escenarios propuesto por Cockburn (2001).

Tabla 7

Escenario del Caso de Uso CU01: Registrar cuenta

Caso de uso	CU01 Registrar cuenta
Actor	Visitante

Requerimientos	RF01
Descripción	Permite que una persona cree su cuenta en el sistema.
Precondición	El visitante se encuentra en la pantalla de acceso.
Flujo básico	1. El visitante selecciona la pestaña «Crear Cuenta». 2. Ingresa el nuevo usuario y la contraseña y pulsa «Registrarse». 3. El sistema valida el formato del usuario y la longitud de la contraseña (RN02). 4. El sistema verifica que el usuario no exista sin distinguir mayúsculas (RN01). 5. El sistema genera el hash bcrypt y registra la cuenta en Supabase. 6. El sistema muestra «Cuenta creada con éxito. Ahora inicia sesión».
Flujos alternos	3a. Formato inválido o contraseña corta: se muestra el error y se solicita corregir. 4a. El usuario ya existe: se informa y no se registra.
Postcondición	La cuenta queda registrada y el visitante puede iniciar sesión.

Tabla 8
Escenario del Caso de Uso CU02: Iniciar sesión

Caso de uso	CU02 Iniciar sesión
Actor	Visitante
Requerimientos	RF02
Descripción	Permite autenticarse para acceder al espacio de trabajo.
Precondición	El visitante tiene una cuenta registrada.
Flujo básico	1. El visitante ingresa su usuario y contraseña. 2. El sistema busca la cuenta sin distinguir mayúsculas. 3. El sistema compara la contraseña con el hash bcrypt. 4. El sistema actualiza la fecha de último acceso. 5. El sistema muestra el Dashboard con el nombre de su espacio de trabajo.
Flujos alternos	2a/3a. Credenciales incorrectas: se muestra «Usuario o contraseña incorrectos». 2b. Supabase no responde: se informa que no se pudo conectar.
Postcondición	El usuario queda autenticado durante la sesión.

Tabla 9
Escenario del Caso de Uso CU03: Cerrar sesión

Caso de uso	CU03 Cerrar sesión
Actor	Usuario registrado
Requerimientos	RF03
Descripción	Permite finalizar la sesión de trabajo.

Precondición	El usuario ha iniciado sesión.
Flujo básico	1. El usuario pulsa «Cerrar Sesión» en la barra lateral. 2. El sistema elimina los datos de la sesión. 3. El sistema muestra la pantalla de acceso.
Flujos alternos	—
Postcondición	La sesión queda cerrada.

Tabla 10
Escenario del Caso de Uso CU04: Gestionar conexiones

Caso de uso	CU04 Gestionar conexiones
Actor	Administrador de BD
Requerimientos	RF04, RF05
Descripción	Permite registrar, actualizar, probar y eliminar las conexiones a los motores.
Precondición	El usuario ha iniciado sesión.
Flujo básico	1. El usuario abre el módulo «Bases de Datos». 2. Selecciona el motor (MongoDB, Redis o Cassandra). 3. Ingresa las credenciales (URI, host y puerto, o Secure Connect Bundle); puede usar «Probar conexión» para verificarlas sin guardar. 4. Pulsa «Probar y guardar» y el sistema prueba la conexión con el motor. 5. El sistema cifra la configuración con Fernet y la guarda en Supabase (RN04). 6. El sistema muestra la conexión como configurada.
Flujos alternos	4a. La prueba falla: se muestra el error y no se guarda. 2a. Eliminar: el usuario pulsa «Eliminar conexión guardada» y el sistema la borra.
Postcondición	La conexión queda guardada cifrada o eliminada.

Tabla 11
Escenario del Caso de Uso CU05: Gestionar reglas de integridad

Caso de uso	CU05 Gestionar reglas de integridad
Actor	Analista de calidad de datos
Requerimientos	RF06, RF07, RF08
Descripción	Permite crear, consultar y eliminar reglas de integridad.
Precondición	El usuario ha iniciado sesión.

Flujo básico	1. El usuario abre el módulo «Reglas» y la pestaña «Crear Nueva Regla». 2. Selecciona el tipo de regla y el motor de origen. 3. Ingresa la colección y el campo; para esquema, el JSON Schema; para referencial o consistencia, el motor, la colección y el campo de destino. 4. El sistema valida los datos obligatorios (RN05 a RN07). 5. El sistema guarda la regla en Supabase y la muestra en «Reglas Activas».
Flujos alternos	4a. Faltan datos o el JSON es inválido: se indica el campo a corregir. 1a. Eliminar: el usuario pulsa «Eliminar» en una regla; el sistema la borra con sus violaciones (RN12).
Postcondición	La regla queda activa o eliminada.

Tabla 12
Escenario del Caso de Uso CU06: Validar registro previo a la escritura

Caso de uso	CU06 Validar registro previo a la escritura
Actor	Desarrollador
Requerimientos	RF09
Descripción	Permite comprobar si un registro cumple las reglas antes de escribirlo.
Precondición	Existen reglas para la colección y una conexión al motor.
Flujo básico	1. El usuario abre «Ejecución» y el simulador previo a la escritura. 2. Selecciona el motor, la colección e ingresa el registro en JSON. 3. El sistema obtiene las reglas de esa colección. 4. El sistema valida el registro contra cada JSON Schema. 5. El sistema consulta al motor si el valor de cada campo único ya existe. 6. El sistema muestra «Validación exitosa».
Flujos alternos	4a. Incumple el esquema: se muestra el error de esquema. 5a. El valor ya existe: se muestra el error de unicidad.
Postcondición	El usuario conoce si el registro puede escribirse.

Tabla 13
Escenario del Caso de Uso CU07: Ejecutar escaneo por lotes

Caso de uso	CU07 Ejecutar escaneo por lotes
Actor	Administrador de BD / Analista
Requerimientos	RF10
Descripción	Evalúa todas las reglas activas sobre los datos existentes.
Precondición	Existen reglas activas y conexiones a los motores involucrados.

Flujo básico	1. El usuario pulsa «Iniciar Escaneo Profundo». 2. El sistema obtiene las reglas activas del usuario. 3. Para cada regla, obtiene los registros de los motores de origen y destino. 4. El sistema evalúa la regla según su tipo (RN08 a RN10). 5. El sistema registra las violaciones (incluye CU08). 6. El sistema muestra la cantidad de anomalías detectadas.
Flujos alternos	3a. Un motor no responde: se registra el error, se conservan las violaciones previas de esa regla y se continúa (RNF06). 6a. Sin violaciones: se muestra «Bases de datos íntegras».
Postcondición	El historial de auditoría refleja el estado actual de los datos.

Tabla 14
Escenario del Caso de Uso CU08: Registrar violaciones

Caso de uso	CU08 Registrar violaciones
Actor	Sistema (incluido en CU07)
Requerimientos	RF11
Descripción	Guarda en la auditoría las violaciones de una regla.
Precondición	Una regla fue evaluada correctamente.
Flujo básico	1. El sistema elimina las violaciones previas de la regla (RN11). 2. El sistema convierte cada dato afectado a JSON. 3. El sistema inserta las violaciones en Supabase con su fecha.
Flujos alternos	—
Postcondición	Las violaciones de la regla quedan actualizadas.

Tabla 15
Escenario del Caso de Uso CU09: Monitorear motores

Caso de uso	CU09 Monitorear motores
Actor	Administrador de BD
Requerimientos	RF13, RF14
Descripción	Muestra el estado y el contenido de un motor.
Precondición	Existe una conexión guardada para el motor.
Flujo básico	1. El usuario abre «Monitor» y selecciona el motor. 2. Pulsa «Ejecutar Diagnóstico Completo». 3. El sistema mide la latencia y obtiene la versión y los recursos. 4. El sistema muestra esquemas, una muestra de registros y estadísticas.
Flujos alternos	3a. Error de conexión: se muestra el mensaje del motor.
Postcondición	El usuario conoce el estado del motor.

Tabla 16

Escenario del Caso de Uso CU10: Consultar dashboard

Caso de uso	CU10 Consultar dashboard
Actor	Auditor interno / Usuario registrado
Requerimientos	RF12
Descripción	Presenta las métricas generales del espacio de trabajo.
Precondición	El usuario ha iniciado sesión.
Flujo básico	1. El usuario abre «Dashboard». 2. El sistema obtiene las reglas y violaciones del usuario. 3. El sistema muestra los motores usados, la cantidad de reglas y anomalías, y el resumen de auditorías.
Flujos alternos	2a. Supabase no responde: se muestra un error y se detiene la página.
Postcondición	—

Tabla 17

Escenario del Caso de Uso CU11: Generar reporte de anomalías

Caso de uso	CU11 Generar reporte de anomalías
Actor	Auditor interno / Analista
Requerimientos	RF15
Descripción	Lista las violaciones con explicación y sugerencia de corrección.
Precondición	Existen violaciones registradas.
Flujo básico	1. El usuario abre «Reportes». 2. El sistema obtiene las violaciones del usuario. 3. Para cada una muestra el identificador del dato, la explicación y la sugerencia según su tipo.
Flujos alternos	2a. Sin violaciones: se informa que no hay anomalías.
Postcondición	El usuario conoce las anomalías y cómo corregirlas.

Tabla 18

Escenario del Caso de Uso CU12: Exportar reporte PDF

Caso de uso	CU12 Exportar reporte PDF
Actor	Auditor interno
Requerimientos	RF16
Descripción	Extiende CU11 para descargar el reporte en PDF.
Precondición	Se está visualizando el reporte de anomalías.

Flujo básico	1. El usuario pulsa «Descargar Reporte PDF». 2. El sistema genera el PDF con fpdf2. 3. El navegador descarga el archivo.
Flujos alternos	—
Postcondición	El usuario dispone del reporte en PDF.

Tabla 19

Escenario del Caso de Uso CU13: Ver usuarios en línea

Caso de uso	CU13 Ver usuarios en línea
Actor	Usuario registrado
Requerimientos	RF17
Descripción	Muestra la comunidad de usuarios conectados.
Precondición	El usuario ha iniciado sesión.
Flujo básico	1. El usuario abre «Comunidad». 2. El sistema obtiene los usuarios con actividad en los últimos cinco minutos (RN13). 3. El sistema muestra la lista de usuarios en línea.
Flujos alternos	—
Postcondición	—

3. Modelo Lógico

a) *Análisis de Objetos*

El análisis de objetos identifica las clases de interfaz, de control y de entidad que realizan los casos de uso, según el enfoque de análisis robusto descrito por Larman (2004). La Tabla 20 resume sus responsabilidades.

Tabla 20

Análisis de Objetos

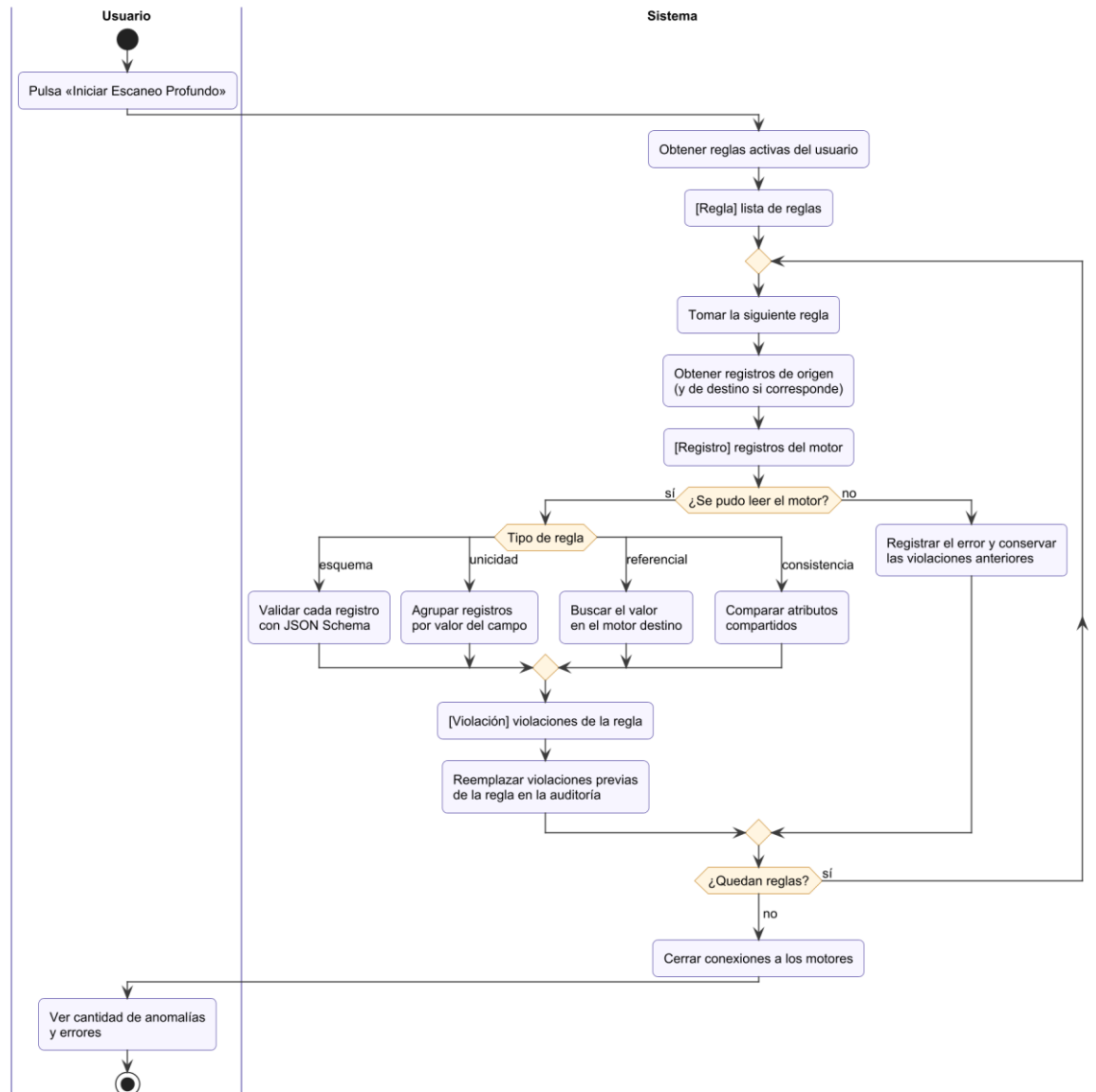
Objeto	Estereotipo	Responsabilidad
Pantalla de acceso	Interfaz	Capturar credenciales para registrar o iniciar sesión.
Página Bases de Datos	Interfaz	Capturar, probar, guardar y eliminar conexiones.
Página Reglas	Interfaz	Crear, listar y eliminar reglas.
Centro de Ejecución	Interfaz	Ejecutar la validación previa y el escaneo por lotes.
Monitor, Dashboard, Reportes y Comunidad	Interfaz	Presentar el estado de los motores, las métricas, las anomalías y los usuarios en línea.

Objeto	Esteretipo	Responsabilidad
Controlador de autenticación	Control	Coordinar el registro, el inicio de sesión y la actividad.
Controlador de reglas	Control	Coordinar la gestión de reglas.
Controlador de verificación	Control	Coordinar el validador y el verificador por lotes.
Controlador de reportes	Control	Obtener las violaciones para el dashboard y los reportes.
Validador	Control	Evaluar un registro contra las reglas de esquema y unicidad.
Verificador por lotes	Control	Evaluar todas las reglas sobre los datos de los motores.
Usuario	Entidad	Representar la cuenta y sus credenciales protegidas.
Conexión	Entidad	Guardar la configuración cifrada de cada motor.
Regla	Entidad	Definir la restricción de integridad y sus parámetros.
Violación	Entidad	Registrar cada incumplimiento detectado.
Reporte	Entidad	Agrupar las violaciones para su presentación y exportación.

Nota. Elaboración propia.

b) Diagrama de Actividades con objetos

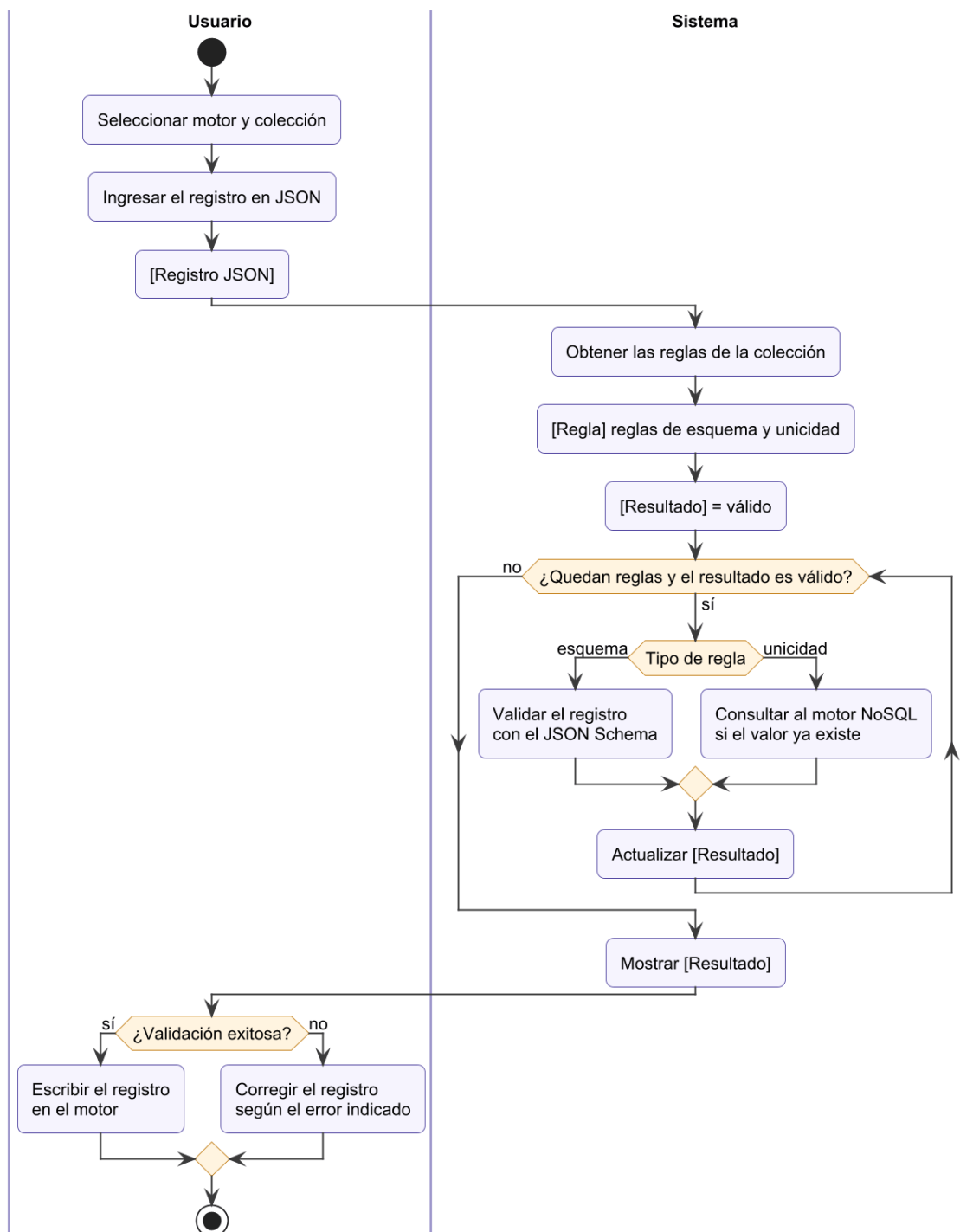
Las Figuras 6 y 7 muestran los dos procesos centrales del sistema con los objetos que se producen y consumen en cada actividad: el escaneo por lotes, en el que las reglas generan violaciones, y la validación previa a la escritura, en la que el registro produce un resultado.

Figura 6
Diagrama de Actividades con Objetos: Escaneo por Lotes


Nota. Elaboración propia con PlantUML (The PlantUML Team, s.f.).

Figura 7

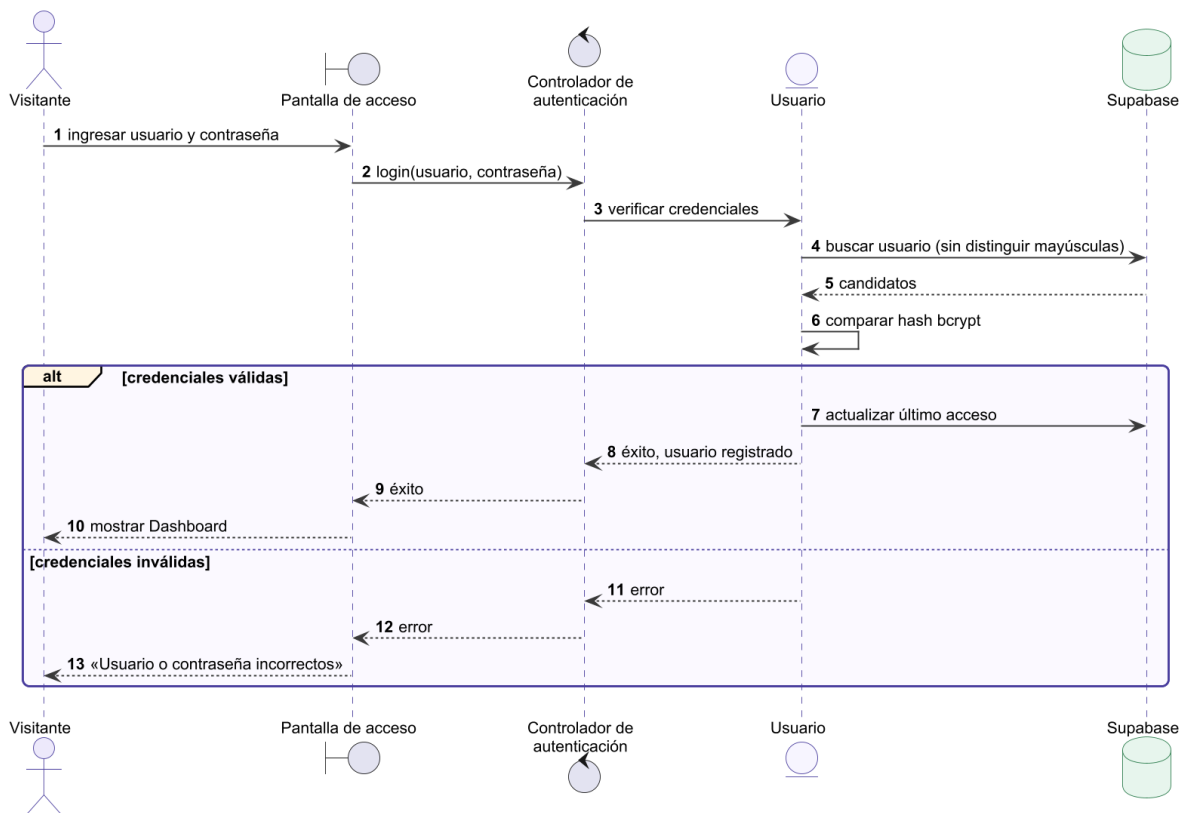
Diagrama de Actividades con Objetos: Validación Previa a la Escritura



Nota. Elaboración propia con PlantUML (The PlantUML Team, s.f.).

c) Diagrama de Secuencia

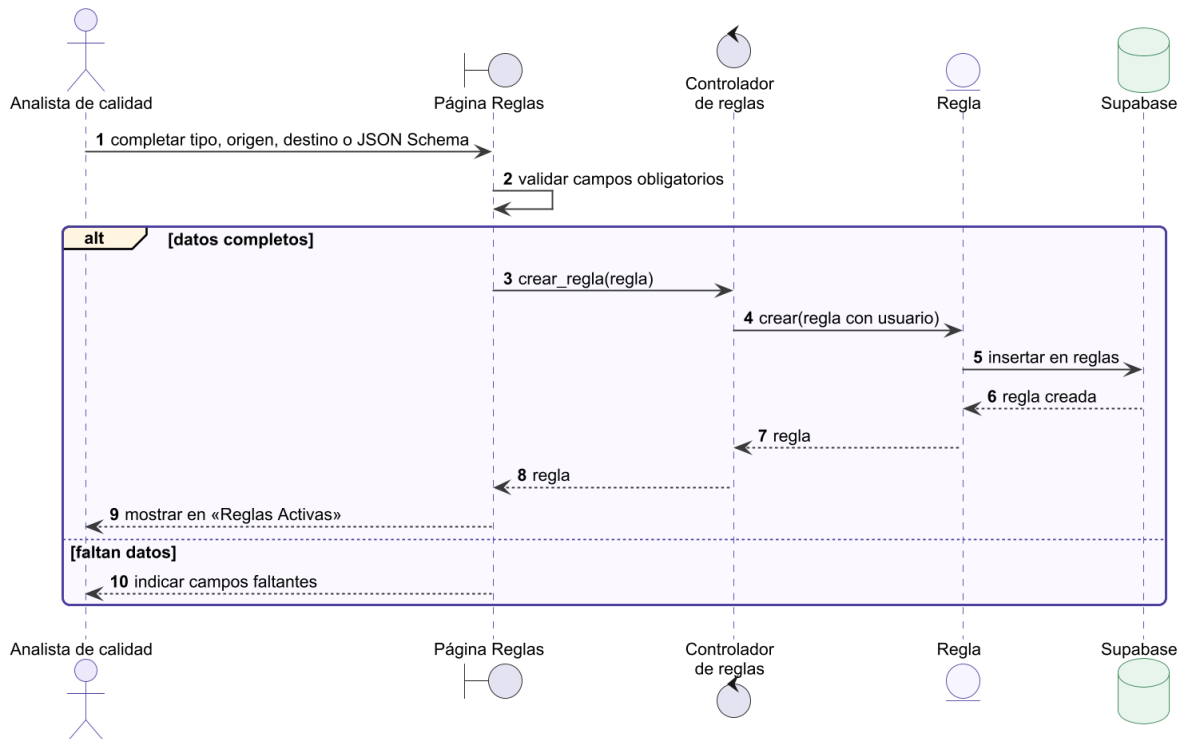
Las Figuras 8 a 11 muestran, a nivel de análisis, el intercambio de mensajes entre los objetos de interfaz, control y entidad en los casos de uso principales.

Figura 8
Diagrama de Secuencia: CU02 Iniciar Sesión


Nota. Elaboración propia con PlantUML (The PlantUML Team, s.f.).

Figura 9

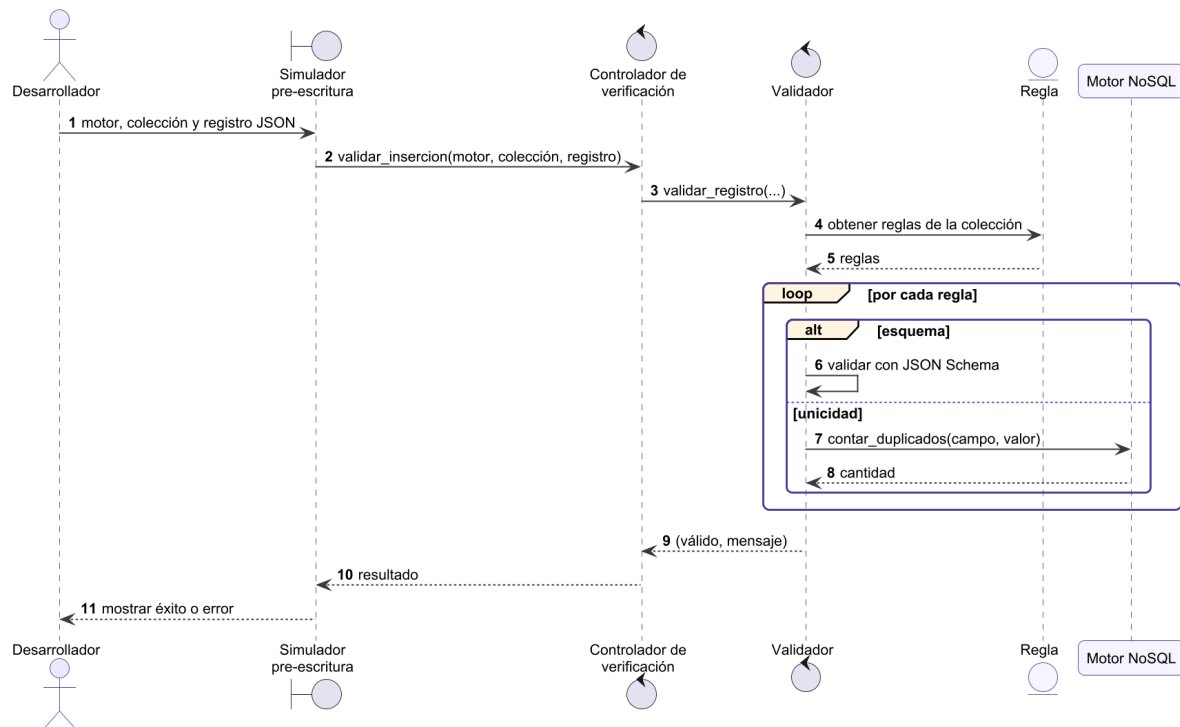
Diagrama de Secuencia: CU05 Gestionar Reglas de Integridad



Nota. Elaboración propia con PlantUML (The PlantUML Team, s.f.).

Figura 10

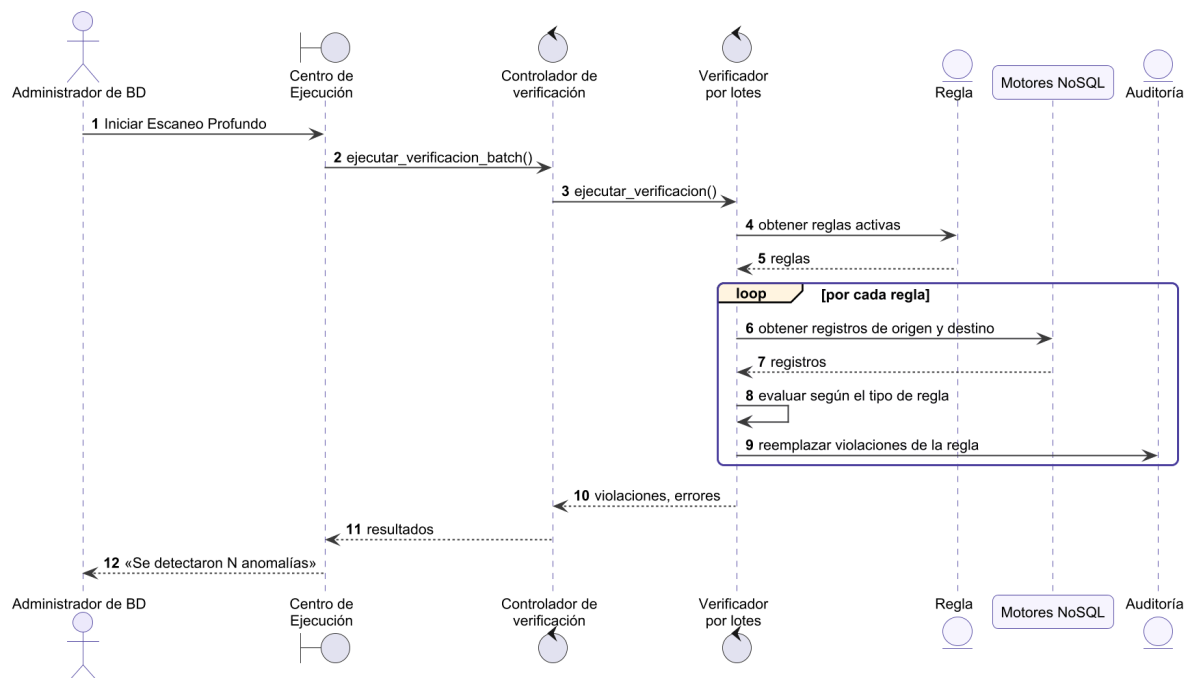
Diagrama de Secuencia: CU06 Validar Registro Previo a la Escritura



Nota. Elaboración propia con PlantUML (The PlantUML Team, s.f.).

Figura 11

Diagrama de Secuencia: CU07 Ejecutar Escaneo por Lotes



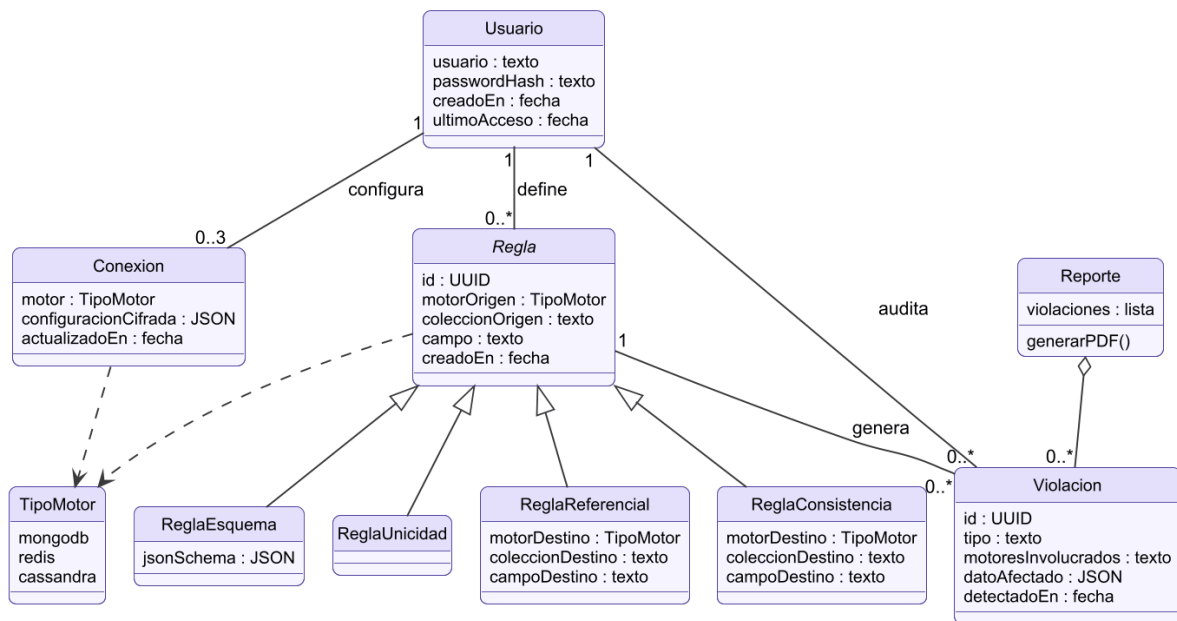
Nota. Elaboración propia con PlantUML (The PlantUML Team, s.f.).

d) Diagrama de Clases

La Figura 12 presenta el diagrama de clases del dominio. La clase abstracta Regla se especializa en los cuatro tipos de regla; cada regla genera violaciones y cada usuario configura hasta tres conexiones, una por motor. El diseño detallado de clases se desarrolla en el documento de arquitectura (FD04).

Figura 12

Diagrama de Clases del Dominio



Nota. Elaboración propia con PlantUML (The PlantUML Team, s.f.).

CONCLUSIONES

Se especificaron 17 requerimientos funcionales, 12 no funcionales y 13 reglas de negocio, derivados de 11 requerimientos iniciales obtenidos con entrevistas, observación, revisión documental y una prueba de concepto.

El análisis de procesos evidenció que el control actual es manual y reactivo, mientras que el proceso propuesto es preventivo y detectivo, y deja evidencia auditable.

Los trece casos de uso y sus narrativas cubren la totalidad de los requerimientos funcionales finales, lo que asegura la trazabilidad entre necesidades, requerimientos y modelos.

El modelo lógico, con sus diagramas de actividades, secuencia y clases, constituye la base para el diseño de la arquitectura descrito en el FD04.

RECOMENDACIONES

- Validar con CloudNexus las narrativas de los casos de uso antes de las pruebas de aceptación.
- Mantener actualizada la matriz de trazabilidad entre requerimientos, casos de uso y pruebas.
- Evaluar en una versión futura la programación automática de escaneos y las alertas por correo.
- Revisar periódicamente los límites de los planes gratuitos frente al crecimiento de los datos del cliente.

BIBLIOGRAFÍA

Cockburn, A. (2001). *Writing effective use cases*. Addison-Wesley.

International Organization for Standardization. (2018). *Systems and software engineering — Life cycle processes — Requirements engineering* (ISO/IEC/IEEE Standard No. 29148:2018).

Larman, C. (2004). *Applying UML and patterns: An introduction to object-oriented analysis and design and iterative development* (3.^a ed.). Prentice Hall.

Sadalage, P. J., y Fowler, M. (2012). *NoSQL distilled: A brief guide to the emerging world of polyglot persistence*. Addison-Wesley.

WEBGRAFÍA

The Apache Software Foundation. (s.f.). *Introduction* [Modelado de datos]. Apache Cassandra Documentation. Recuperado el 3 de octubre de 2026, de <https://cassandra.apache.org/doc/latest/cassandra/developing/data-modeling/intro.html>

MongoDB, Inc. (s.f.). *Schema validation*. MongoDB Docs. Recuperado el 3 de octubre de 2026, de <https://www.mongodb.com/docs/manual/core/schema-validation/>

Object Management Group. (2017). *OMG Unified Modeling Language (OMG UML)* (Versión 2.5.1). <https://www.omg.org/spec/UML/2.5.1>

The PlantUML Team. (s.f.). *PlantUML*. Recuperado el 3 de octubre de 2026, de <https://plantuml.com/>

Redis Ltd. (s.f.). *Redis data types*. Redis Docs. Recuperado el 3 de octubre de 2026, de <https://redis.io/docs/latest/develop/data-types/>